A Two-Stage Entity Resolution Pipeline with FAISS Blocking and Splink Matching

Denis Robert

August 2026

Abstract

This paper presents and evaluates a two-stage entity resolution pipeline for incomplete Canadian person records. A synthetic reference population is represented with normalized `all-MiniLM-L6-v2` embeddings and persisted in a FAISS `IndexFlatIP` store. Each query is first blocked to its top 20 vector neighbors, after which Splink performs interpretable probabilistic linkage using Jaro–Winkler comparisons for names and address, a dedicated email comparison, and a date-of-birth comparison that accounts for date structure. The approach is similar in overall architecture to the FAISS-plus-linkage workflow described by Strojny and Beręsewicz [6], although the present pipeline was independently derived and uses a different implementation and evaluation design. The implementation supports persisted-index reload without re-embedding the reference population and independently models 30% missingness for address and email. In a 15,000-query experiment comprising identical, close-variation, and unrelated records, the updated pipeline achieved 99.28% accuracy, 99.51% precision, 99.41% recall, 99.02% specificity, and a 99.46% F1 score, with a mean query time of 7.48 ms in the validation environment. The pipeline is designed for fast online deduplication in which the operator knows the desired cost profile in advance — the relative cost of a false positive versus a false negative — so that the decision threshold can be set cost-optimally. The paper also discusses memory scaling, operational limits, calibration of Splink parameters, and alternatives to linear-scan FAISS indexing.

1 Introduction

Entity resolution attempts to determine whether two records refer to the same real-world entity. This task is difficult when records contain typographical variation, missing values, stale addresses, or inconsistent contact information. Comparing every query against every reference record is also unnecessarily expensive: for N reference records, exhaustive comparison requires $O(N)$ candidate comparisons per query.

The implementation in the entity-resolution repository addresses this problem with a retrieval-and-linkage architecture. A dense vector index [9] performs approximate or exact nearest-neighbor retrieval, reducing the candidate set to the closest k records. This use of dense retrieval for blocking follows a line of work including DeepER [5], pre-trained embedding comparisons [4], and universal dense blocking [3]. Splink then evaluates those candidates with explicit field comparisons and returns a match only when its probability exceeds a configurable threshold.

The representation choice is informed by recent work comparing tabular representation approaches. Vogel et al.'s *Towards Universal Tabular Embeddings: A Benchmark Across Data Tasks* (arXiv:2604.21696v1, `https://arxiv.org/abs/2604.21696v1`) [8] benchmark embeddings at cell, row, column, and table levels and show that performance depends on the downstream task and

representation level; in the relevant current evaluation, textual embeddings outperform the dedicated tabular alternatives for retrieval. Franz et al.'s *Universal Embeddings of Tabular Data* (arXiv:2507.05904v1, https://arxiv.org/abs/2507.05904v1) [7] describes a different strategy that transforms tabular data into a graph, learns entity embeddings with graph auto-encoders, and aggregates them into row embeddings. This graph-based approach may bear fruit for entity resolution in future experiments because it is designed to capture relationships among tabular entities and to embed unseen samples composed of similar entities. The present system therefore uses a row-level textual representation as its current baseline while leaving room to replace the embedding engine as these alternatives become more capable.

2 System Objective and Data Model

The reference population contains 50,000 synthetic Canadian person records. Each record has:

- first name;
- last name;
- date of birth;
- a Canadian address; and
- an email address.

Address and email are independently removed with a configurable default probability of 0.30. Consequently, first name, last name, and date of birth are the most consistently available identity attributes, while address and email are useful but incomplete evidence. Addresses are also treated as less stable because people move and formatting conventions change.

A note on preprocessing: the problem that inspired this research supplied person records with pre-parsed first and last name fields and an address field already standardized against a postal reference database, so the pipeline required no name or address segmentation and no normalization preprocessing before embedding and linkage; it consumes structured fields as given. The relaxation of that assumption, and its accuracy and resource implications, are examined in the parsing-and-segmentation research item of Section 10.6.

A record is serialized into a structured text string:

```
First Name: ...
Last Name: ...
Date of Birth: ...
Address: ...
Email: ...
```

Missing fields are omitted from the embedding text and retained as null metadata. The same person object is used for the query and for the persisted metadata, avoiding a second source of truth.

2.1 Row serialization strategies

The embedding ablation compares three textual representations of the same structured record. These representations affect only dense retrieval; Splink receives the original structured fields and uses the same comparisons in every experiment. For example, consider the record

```
first_name    = Alice
last_name     = Wong
date_of_birth = 1985-06-15
address       = 123 Main Street, Toronto, ON M5V 1A1
email         = alice.wong@example.com
```

The **default** strategy is the production representation implemented by `Person.to_text`. It uses labelled lines in identity, address, and email order:

```
First Name: Alice
Last Name: Wong
Date of Birth: 1985-06-15
Address: 123 Main Street, Toronto, ON M5V 1A1
Email: alice.wong@example.com
```

The **identity-first** strategy retains the same labels and values but places the optional contact fields in email-then-address order after the identity fields:

```
First Name: Alice
Last Name: Wong
Date of Birth: 1985-06-15
Email: alice.wong@example.com
Address: 123 Main Street, Toronto, ON M5V 1A1
```

This tests whether giving the embedding model an explicit identity prefix changes retrieval, without changing the vocabulary or the underlying data. The **compact** strategy removes labels and joins the fixed field sequence with pipe delimiters:

```
Alice|Wong|1985-06-15|123 Main Street, Toronto, ON M5V 1A1|alice.wong@example.com
```

For a record with missing address and email, the labelled strategies omit those lines, whereas compact serialization preserves field positions with empty values:

```
First Name: Alice
Last Name: Wong
Date of Birth: 1985-06-15

Alice|Wong|1985-06-15||
```

The default strategy is the baseline used by the resolver and confusion-matrix experiment. Identity-first and compact are evaluation alternatives, not separate linkage models. Comparing them under the same FAISS index type, blocking sizes, Splink configuration, queries, and thresholds isolates the effect of row representation on candidate recall and downstream matching.

3 Two-Stage Algorithm

3.1 Offline generation and indexing

The offline process in `generate_data.py` performs the following steps:

1. Generate the reference population with a fixed random seed when reproducibility is required.
2. Convert every person into its textual row representation.
3. Compute a dense vector with `sentence-transformers/all-MiniLM-L6-v2` through `HuggingFaceEmbeddings`.
4. L2-normalize each vector and add it to a FAISS `IndexFlatIP` index. Inner product on normalized vectors is cosine similarity.
5. Persist the FAISS index as `people.faiss` and the person records, model name, and normalization setting as `people.json`.

The metadata file is deliberately separate from the FAISS binary index. FAISS stores the vectors and their positional order; the JSON file stores the record associated with each position. Loading reconstructs the embedding client and reuses the saved vectors without re-embedding the reference population.

3.2 Online blocking

Given a query record q, the online process embeds the same textual representation and searches the index for the top k records. The default is $k = 20$. If $e(q)$ and $e(r)$ are normalized query and reference vectors, the blocking score is

$$s_{\text{FAISS}}(q, r) = e(q)^T e(r),$$

which is cosine similarity in the range $[-1, 1]$. This stage is optimized for recall: the true match must be included in the candidate set for the second stage to recover it. FAISS [9] provides this nearest-neighbour search.

3.3 Temporal-aware blocking for records with known capture dates

The blocking score above weights every retrieved reference equally. When reference records carry a known capture (creation) date, this should be exploited at blocking time, because the *relevance of each field to the match decision decays with the age gap* between two records. The address is the clearest case: it is stable within a short window (two records captured near each other at the same address) but becomes actively misleading once records are more than roughly the typical residency period apart, when the same entity has since *moved*. Names change only in controlled ways (nicknames, abbreviations, marriage) and date of birth is invariant, so these form the reliable long-gap anchors.

For a query q and reference r, let $\Delta t_{q,r}$ be their capture-date gap and T the residency timescale of the deployment. The recommendation for a multi-index blocker is to age each field's retrieval score continuously rather than apply a single fixed weight:

$$s_{\text{block}}(q, r) = s_{\text{identity}}(q, r) + e^{-\Delta t_{q,r}/T}\, s_{\text{address}}(q, r), \tag{1}$$

where s_{identity} is the retrieval score on the invariant fields (first, last, date of birth) and s_{address} the score on the address view. The identity score always counts at full weight; the address score is exponentially down-weighted as records age apart. Because $\Delta t_{q,r}$ is a pair-wise quantity, the weight is applied at *score-fusion time* (after retrieving each view) rather than baked into a precomputed vector.

Measured on the synthetic population with a controlled age gap (6,000-record index, 180 duplicate pairs, T=10 years), this recovers the temporal failure mode: in the long-gap case the

address-only "contact" view collapses to 0.22 recall at k=1, while the gap-weighted score recovers 0.99 at k=1 and 1.0 by k=5 — comparable to relying on invariant identity alone, and far above the unweighted single view. In the short-gap case it also reaches 1.0 by k=5, at the cost of a small k=1 penalty against the address-rich "full" view (0.91 versus 1.00), which disappears in the working range ($k \geq 5$). The details and full table appear in the companion paper on batch deduplication [28]; for insertion-time blocking with known capture dates the same decay should be used. When capture dates are unavailable, or when the population is effectively co-temporal (all records sharing one generation date, as in the synthetic experiments of Section 8), the temporal caveat does not apply and the single-vector "full" index suffices. The decay rate T must be set from the deployment's observed residency distribution rather than assumed.

A note on stage: the decay in (1) acts at *blocking* time — it attenuates a cosine *retrieval* score during candidate selection and leaves the comparison model inside the Fellegi–Sunter matcher time-independent. The temporal record-linkage literature instead applies decay to the comparison weights; the relationship between that line of work and the present design, and why the blocking-side decay cannot be absorbed into m/u, is treated in the companion design note [24] and in the accompanying literature survey [25].

3.4 Probabilistic linkage

The candidate set is passed to Splink as a small deduplication problem containing the query and the retrieved records. Splink implements a scalable form of probabilistic linkage grounded in the Fellegi–Sunter framework [1, 2]. The configured comparisons and their intended evidence priority are listed in Table 1:

Table 1: Field comparisons and intended evidence priority.

Field	Comparison	Intended role
First name	Jaro–Winkler thresholds	High-priority identity evidence
Last name	Jaro–Winkler thresholds	High-priority identity evidence
Date of birth	Date-of-birth comparison	Strong, stable evidence
Email	Email comparison	Medium-priority evidence when present
Address	Jaro–Winkler thresholds	Lower-priority, changeable evidence

Splink produces a match probability from the comparison vector and its m/u parameters. The resolver selects the highest-scoring candidate involving the query and returns it only when

$$P(\text{match} \mid \text{comparisons}) \geq \tau,$$

where τ is the configured match threshold. Missing values are not treated as negative evidence; they provide no comparison agreement and the remaining fields determine the result.

The implementation uses FAISS for blocking and Splink 4.x inference for linkage. The current settings provide comparison ordering and thresholded similarity levels. Ideally, in a production deployment, Splink's m/u probabilities should be trained on representative labelled pairs rather than relying on defaults.

A note on the linkage side of the decay. The piecewise version of the temporal decay is already expressible inside Splink without any new mechanism: Robin Linacre notes in a public comment on Splink issue #3240 [26] that the address comparison can be bucketed by the capture-date gap into comparison levels, such as "same address, capture dates at most one year apart," "same address,

one to five years apart," "same address, five to ten years apart," "same address, more than ten years apart," and "different address." Each level carries its own m/u probabilities and therefore its own match weight, and address disagreement can be stratified by time gap in the same way. This yields a piecewise approximation to the continuous decay of (1) without adding a weighting mechanism to Splink and without committing to a specific parametric decay function — at the cost of committing to a fixed set of bucket boundaries. The fidelity of the approximation is set by the bucket widths: refining them moves the step function toward the smooth entropy-loss weight $w_c(\Delta t)$ derived in the companion literature survey [25], at the price of needing enough training or estimated pairs in every bucket for its m/u to be stable. The measured pipeline in this whitepaper uses the retrieval-time fusion of (1) and keeps the linkage model time-independent, but deployments already estimating per-level m/u in Splink can obtain the same qualitative effect by this bucketing route.

4 Persistence and Operational Workflow

The basic workflow. The pipeline is operated with two explicit commands, an offline build and an online query:

```
python scripts/generate_data.py --count 50000 --missing-rate 0.3 --output-dir data
python scripts/search_cli.py --index-dir data --input-json query.json --threshold 0.85
```

The first command is the offline build step of Section 3.1. It generates a reference population of 50,000 synthetic Canadian person records with address and email independently missing at a default rate of 0.3 — serializes every record into the textual row representation of Section 2.2, embeds the rows with `sentence-transformers/all-MiniLM-L6-v2` through `HuggingFaceEmbeddings`, L2-normalizes the vectors, and adds them to a FAISS `IndexFlatIP` index. The artifact is persisted as two files: the binary index `people.faiss`, which holds the normalized vectors in positional order, and `people.json`, which stores the person record, model name, and normalization setting associated with each position. Keeping the files separate is deliberate (Section 3.1): the vector store answers nearest-neighbour queries, the metadata file supplies the record behind every position, and a reload reuses the saved vectors without re-embedding the reference population.

The second command is the online query path. It loads the persisted pair of files, reconstructs the embedding client from the saved model name, and reuses the stored vectors without re-embedding the reference population. Given an input person in `query.json`, it applies the same textual row representation used at build time, embeds the query record, and searches the index for the top 20 candidates under the default block size (Section 3.2). The candidate set is passed to Splink as a small deduplication problem of one query against its retrieved records, using the comparisons listed in Table 1; the command then either emits a match — the highest-scoring candidate together with its Splink probability and the matched metadata — when that probability reaches the configured threshold τ, or a no-match decision when it does not. Separating the two commands prevents the expensive population embedding and indexing from being repeated on every query and makes the persisted index a deployable artifact.

5 FAISS Applicability and Memory Bounds

The 50,000-record artifact generated by this implementation provides a concrete sizing baseline. On disk, the measured `people.faiss` file is 76.8 MB and the `people.json` metadata file is 10.5 MB, for a combined 87.3 MB. This corresponds to approximately 1,536 bytes per normalized 384-dimensional float32 vector, 210 bytes per serialized person record, and 1,746 bytes per record

including both artifacts. The index dimension follows the MiniLM embedding model used by the baseline.

The following extrapolation assumes the same embedding dimension, float32 storage, flat inner-product index, average metadata length, and one metadata record per vector. It uses binary RAM units (1 GiB = 2^{30} bytes), although the table labels them as the commonly used 16, 32, and 128 GB deployment sizes. It excludes the Python interpreter, embedding model, temporary construction buffers, allocator fragmentation, operating-system memory, and concurrent queries. Table 2 summarises the resulting capacity bounds.

Table 2: Estimated capacity when the FAISS index and person metadata share the available RAM.

Available RAM	Raw capacity	Index size	Metadata size	Recommended*
16 GB	9.84 million	14.4 GB	2.0 GB	6.89 million
32 GB	19.68 million	28.9 GB	4.1 GB	13.77 million
128 GB	78.71 million	115.3 GB	16.5 GB	55.10 million

*Recommended capacity reserves 30% of RAM for the operating system, Python process, model weights, FAISS temporary allocations, and query concurrency. The raw capacity is a sizing upper bound, not a production recommendation. During generation, peak memory can be higher because embeddings may be materialized before being added to FAISS. Building large indexes should therefore use batches and measure peak resident set size rather than relying only on final file size.

For this flat `IndexFlatIP` design, a query scans all vectors, so memory capacity and query latency are coupled to the record count. At tens of millions of records, an approximate index such as HNSW or an inverted-file/product-quantized index should be evaluated; FAISS [9] ships several such indexes. Quantization can reduce memory substantially, but may reduce blocking recall and must be measured against the downstream Splink match rate. Regardless of the FAISS index type, the metadata mapping must preserve vector-to-person row order and should be loaded with equivalent memory budgeting.

5.1 When an external vector database becomes appropriate

An external vector database is not automatically better than FAISS. For a single service instance, a stable reference population, and a dataset that fits comfortably in memory, local FAISS has fewer moving parts and avoids network latency. The decision should be based on operational bounds as well as raw capacity. With the measured baseline, 10 million records would require approximately 17.5 GB for the flat index plus metadata, while 50 million records would require approximately 87.3 GB before runtime overhead. These sizes place a single-process deployment near or beyond the recommended envelope of a 16 or 32 GB host and make a 128 GB host increasingly specialized.

An external vector database is likely to become more appropriate under any of the following conditions:

- the working set approaches roughly 70% of host RAM, leaving insufficient room for the embedding model, Python, Splink, index rebuilds, and concurrent queries;
- the reference population is large enough that flat-index scan latency or index-build time violates the service-level objective, even after testing FAISS approximate indexes;
- multiple application instances need a shared index, replicas, rolling updates, or failover without copying a large binary artifact to every host;

- records require frequent inserts, deletes, or metadata updates, making immutable local snapshots operationally expensive; or
- the system needs managed durability, access control, monitoring, backups, multi-tenant isolation, or independent scaling of storage and query capacity.

As a practical boundary, local FAISS is a strong default for the 50,000-record baseline and remains straightforward through the low millions. Between roughly 7 million and 14 million records, the 16/32 GB sizing envelope suggests benchmarking an approximate FAISS index against a managed or self-hosted vector database. Above roughly 50 million records, or whenever the index must be shared across services and regions, an external system is generally the safer operational choice. These are planning thresholds rather than algorithmic limits: compression, sharding, and hardware can move them, while a strict latency or availability requirement can move the decision earlier. The external database should still be evaluated by blocking recall at $k = 20$, not only by vector-query latency, because Splink cannot recover a true match that the retrieval layer omits.

6 Complexity and Failure Modes

Let d be the embedding dimension, N the reference population size, and k the blocking size. Building the flat FAISS index costs approximately $O(Nd)$ vector storage and embedding work. A flat inner-product query costs $O(Nd)$, although FAISS offers approximate indexes when the population grows beyond the intended scale. Splink then operates on only $k + 1$ records, making the linkage stage independent of the full population size after blocking.

The principal failure mode is blocking recall. A correct record omitted from the top k candidates cannot be recovered by Splink. Other risks include embedding representations that overemphasize address tokens, duplicate or common names, poorly calibrated Splink probabilities, and distribution shift between synthetic data and operational data. Evaluation should therefore report blocking recall separately from final precision, recall, and calibration.

7 Evaluation Plan

A useful benchmark should contain labelled positive and negative pairs, controlled missingness, realistic address changes, spelling errors, and hard negatives sharing names or dates. The following metrics should be reported:

1. blocking recall at $k \in \{10, 20, 50, 100\}$;
2. final precision, recall, F1, and false match rate at several thresholds;
3. precision–recall and calibration curves for Splink probabilities;
4. mean and tail query latency; and
5. index size and offline build time.

The benchmark should include ablations for missing email, missing address, both fields missing, address changes, and name perturbations. It should also compare row serialization strategies so that improvements are attributable to representations rather than only to the matcher.

7.1 Measured Section 7 benchmark

These requirements were implemented in `experiment_section7_eval.py` and run with 5,000 reference records, 15,000 labelled queries, a 30% independent missingness rate, and 500 records per ablation. The labelled queries consisted of identical positives, close same-entity variants, and unrelated negatives. Splink used its current untrained default m/u parameters, so the measurements are an engineering baseline rather than calibrated production probabilities. The evaluator wrote the complete results to `section7_results.json` and `section7_metrics.csv`.

Table 3 compares the three row serialization strategies. Blocking recall is measured over the 10,000 positive queries. Final linkage metrics use the production top-20 candidate set, while recall is also reported at $k = 10, 50, 100$. Latency percentiles cover embedding plus FAISS blocking per query; the reported end-to-end mean adds the batched Splink inference time divided by the number of queries.

Table 3: Measured serialization and retrieval/linkage results (15,000 queries; artifact `results/erwhitepaper/section7_results.json`).

Strategy	R@10	R@20	R@50	R@100	$P_{.85}$	$R_{.85}$	$F1_{.85}$
Default	99.81%	99.88%	99.95%	99.96%	99.51%	99.85%	99.68%
Identity first	99.81%	99.85%	99.95%	99.96%	99.49%	99.82%	99.66%
Compact	99.93%	99.95%	99.96%	99.98%	99.53%	99.92%	99.73%

The compact serialization was the strongest of the three in this run, improving top-20 blocking recall from 99.88% to 99.95% and reducing the false-match rate at threshold 0.85 from 0.98% to 0.94%. The difference is descriptive rather than statistically conclusive and should be retested on labelled operational data. For the default strategy, index build time was 35.7 seconds, p95 blocking latency was 25.5 ms, and estimated mean end-to-end latency was 20.2 ms. The corresponding values for identity-first were 39.4 seconds, 24.4 ms, and 20.0 ms; compact was 30.8 seconds, 20.8 ms, and 17.1 ms.

Table 4: Blocking-recall ablations at $n = 500$ per condition (artifact `results/erwhitepaper/section7_results.json`).

Condition	R@10	R@20	R@50	R@100
Baseline	100.0%	100.0%	100.0%	100.0%
Missing email	100.0%	100.0%	100.0%	100.0%
Missing address	99.8%	100.0%	100.0%	100.0%
Missing both	98.8%	99.8%	100.0%	100.0%
Address change	100.0%	100.0%	100.0%	100.0%
Name perturbation	100.0%	100.0%	100.0%	100.0%

Table 4 summarises the blocking-recall ablations. At the default 0.85 threshold, every ablation query was accepted in this synthetic positive-only slice once it reached Splink, so the main observed degradation was retrieval: removing both optional fields reduced top-10 recall to 98.8%, recovering to 99.8% by top 20 and 100% by top 50. The evaluator also produces ten-bin reliability data and threshold curves; those outputs should be recalculated after Splink is trained on labelled pairs.

8 Confusion-Matrix Experiment

The implementation includes `experiment_confusion_matrix.py`, which evaluates the current algorithm on $n = 5{,}000$ generated reference rows. For every reference person, the test creates three queries: an identical row labelled as the same entity, a close but non-identical row generated with controlled perturbations and labelled as the same entity, and an independently generated unrelated row labelled as a different entity. The latter is not present in the reference index. FAISS retrieves the top 20 candidates for every query, after which Splink applies the same field comparisons and decision threshold used by the resolver. The experiment batches the queries into one Splink link-only inference call for execution efficiency; this changes execution strategy but not the candidate pairs or scoring configuration. The experiment's metaparameters are listed in Table 5.

Table 5: Confusion-matrix experiment metaparameters.

Parameter	Value
Reference/test rows (n)	5,000
Queries per test row	3
Total queries	15,000
Missing address/email rate	0.30
Embedding model	`all-MiniLM-L6-v2`
FAISS blocking size (k)	20
Splink match threshold (τ)	0.85
Close-row variation rate	0.15
Random seed	42

Table 6: Measured confusion matrix over 15,000 queries (artifact `results/erwhitepaper/confusion_matrix_results.json`).

	Predicted match	Predicted no match
Actual same entity	9,941 (TP)	59 (FN)
Actual different entity	49 (FP)	4,951 (TN)

Table 6 summarises the measured confusion matrix. All 5,000 identical queries were correctly matched. Among the 5,000 close same-entity queries, 4,941 were matched and 59 were missed. Among the 5,000 unrelated queries, 49 were incorrectly accepted and 4,951 were rejected. The resulting accuracy was 99.28%, precision 99.51%, recall 99.41%, specificity 99.02%, and F1 99.46%. Index construction took 40.2 seconds and batched query processing took 112.2 seconds, or 7.48 milliseconds per query on the test environment. These measurements are a baseline rather than a general performance guarantee; hardware, model caching, and Splink configuration affect both quality and latency.

The confusion-matrix protocol is rerun with a fixed seed (42) and, at this scale, its results are reproducible: repeated runs of `experiment_confusion_matrix.py` under the same settings returned the same confusion matrix. We note, however, that Splink's inference runs on DuckDB with parallel workers, so at much larger scales or under a different backend the exact counts can shift by a few in ten thousand; tables should therefore always be read against the persisted artifact named in the caption rather than remembered as exact round numbers.

8.1 Trained versus untrained linkage parameters

All figures reported so far use Splink's default, untrained m/u values together with a fixed match prior of 0.0001. Because the whitepaper's recommendations depend on the measured operating point, we quantify whether calibrating m/u changes the results materially. We reuse the 50,000-record reference index and evaluate the same three-query-per-row protocol for 2,000 rows (6,000 queries) at $\tau = 0.85$ and $k = 20$, holding the match prior at 0.0001 so that the comparison isolates the effect of m/u alone.

Two calibration schemes were considered. The first is Splink's unsupervised expectation maximisation over the reference population. Candidate pairs for EM are generated by exact-equality blocking on two keys, `first_name` and `date_of_birth` (`block_on("first_name")` and `block_on("date_of_birth")`, i.e. the SQL conditions l.first_name = r.first_name and l.date_of_birth = r.date_of_birth), and the same two keys are used to estimate the match prior. Because the reference set is de-duplicated and contains essentially no true duplicate pairs, EM fits m/u from "different people who happen to share a name or date of birth," which inflates match probabilities and yields gross over-matching: on the 100,000-record duplicate-bearing benchmark the EM variant falls to 71.7% F1 (Table 9), and a near-duplicate-free reference offers even less recovery pressure. This negative result is an important caveat: unsupervised EM on a near-duplicate-free reference is not a safe calibration strategy for streaming link-only resolution.

The second, recommended scheme calibrates each comparison level's m and u directly from labelled pairs generated by the same process as the confusion experiment: identical rows and close-variant rows are labelled matches, and independently generated unrelated rows are labelled non-matches. Empirical level probabilities are computed over these pairs with Laplace smoothing (0.5), and null levels are left to Splink's data-driven handling. This supervised calibration is defensible because it uses genuine match and non-match evidence rather than the accidental shared-field structure of the reference population.

Table 7: Trained versus untrained linkage parameters over 50,000 reference records and 6,000 queries ($\tau = 0.85$, $k = 20$, prior 0.0001; artifact `results/erwhitepaper/mu_calibration_results.json`).

Variant	Accuracy	Precision	Recall	Specificity	F1
Untrained (default m/u)	97.50%	97.16%	99.15%	94.20%	98.14%
Trained (supervised, labelled pairs)	91.65%	89.43%	99.20%	76.55%	94.06%

Table 7 compares the two variants at the fixed operating point. The supervised calibration is well behaved and changes the operating point in a measurable way. Recall is essentially unchanged (99.15% to 99.20%), while precision falls from 97.16% to 89.43% and specificity from 94.20% to 76.55%, giving an F1 of 94.06% versus 98.14% untrained. In absolute terms the trained variant trades a small reduction in false negatives (34 to 32) for a substantially larger increase in false positives (116 to 469). The results therefore confirm that reported figures are sensitive to how m/u are obtained, and they motivate the labelled-pair calibration and related research directions discussed in Section 10. As a rule, parameters should be trained on labelled operationally representative pairs and evaluated for calibration before reported numbers are treated as production expectations.

These results also raise a calibration paradox: refining m/u produced worse F1 than leaving them untrained. `experiment_mu_tau_interaction.py` decomposes the paradox on the duplicate-bearing population (5,000 base records, 3% twins; artifact `results/erwhitepaper/mu_tau_interaction.json`) into two distinct mechanisms. First, the untrained parameters are not tuned to any specific corpus.

The pipeline's untrained defaults are a constructed, internally consistent weight ladder — an exact match contributes +10 bits, a full mismatch −5 bits, the exact-match m is set to 0.95, and the default prior is 0.0001. Retraining m/u changes the evidence weights while the prior and the decision threshold stay at their old values, shifting the whole score distribution: the mean match probability of non-match pairs rose from 0.046 (untrained) to 0.285 (supervised) and 0.536 (EM), with the fitted EM prior (0.0071 versus 0.0001) the dominant driver. Here too the EM candidate pairs were generated by exact-equality blocking on the `first_name` and `date_of_birth` keys. Jointly re-optimising the prior and threshold strongly favours the trained model only at a low prior and a high threshold, whereas the untrained defaults are robust across the whole operating-point region, as Table 8 shows. The prior/threshold coupling therefore explains most of the apparent gap — EM at the naive fixed point is poor, and re-tuning the prior and τ recovers a large part of the difference — but it is not the whole story. Second, even after jointly optimising (τ, prior) for each model, the trained m/u stay below the defaults (EM optimum F1 0.955 versus 0.990 untrained), because the trained models assign more weight to the partial-similarity levels that non-match pairs also exhibit. The practical lesson is that the default configuration is a coherent bundle: any change to m/u should be accompanied by re-estimation of the prior and re-selection of τ on a validation set, otherwise the reported metrics are measured at an unintended operating point. Because F1 alone does not separate ranking discrimination from probabilistic calibration, the companion analysis [27] quantifies the same configurations with pairwise Brier and Average Precision (and expected cost): training that sharpens ranking (higher Average Precision on the synthetic benchmark) can nonetheless worsen calibration (Brier) and the fixed-threshold F1, so F1 should be read together with calibration and cost metrics.

Table 8: Joint decision behaviour of F1 (percent) over the match prior and threshold τ, for the untrained and EM-trained m/u on the duplicate-bearing population (5,000 base records, 3% twins; artifact `results/erwhitepaper/mu_prior_tau_surface.json`). The untrained defaults hold a high-F1 band across the region; the EM model matches only at a low prior and high threshold.

Prior	Untrained F1			EM-trained F1		
	τ=0.85	τ=0.90	τ=0.95	τ=0.85	τ=0.90	τ=0.95
10^{-4}	98.3	98.3	99.0	91.4	94.0	95.2
10^{-3}	97.1	97.1	96.4	82.3	82.3	85.1
10^{-2}	96.1	96.1	97.1	81.9	81.9	82.3

8.2 Large duplicate-bearing benchmark at 100K rows

The experiments above evaluate a near-duplicate-free reference. To stress the resolver under a realistic mix of true duplicates and unrelated records, `experiment_duplicate_benchmark.py` builds a 100,000-record base population and adds a near-duplicate twin (small field differences at the same 0.15 variation rate) for 3% of base records, yielding a 103,000-record reference index that genuinely contains matching records. The evaluation uses 6,000 balanced queries: 3,000 positives are fresh small-difference variants that are *not verbatim* in the index but each match a base record, and 3,000 negatives are independently generated unrelated people. Queries are blocked to the top 20 FAISS neighbours and scored with Splink at $\tau = 0.85$; only the linkage parameters differ. Three schemes are compared: the untrained default m/u with a fixed prior of 0.0001; supervised calibration of m/u from labelled duplicate/non-match pairs; and unsupervised expectation maximisation over the reference population, which here contains genuine duplicate pairs for EM to exploit. As in

the calibration section, the EM candidate pairs are generated by exact-equality blocking on the `first_name` and `date_of_birth` keys. The results are compared in Table 9.

Table 9: Large duplicate-bearing benchmark: 100,000 base records plus 3,000 duplicates (103,000-record index) and 6,000 queries ($\tau = 0.85$, $k = 20$; artifact `results/erwhitepaper/training_results.json`).

Variant	Accuracy	Precision	Recall	Specificity	F1
Untrained (default m/u, prior 0.0001)	93.33%	90.12%	97.33%	89.33%	93.59%
Supervised (labelled pairs)	82.58%	75.03%	97.67%	67.50%	84.87%
Unsupervised (EM, prior 0.0071)	61.52%	56.69%	97.57%	25.47%	71.71%

In confusion-matrix terms the untrained variant produced TP = 2,920, FN = 80, FP = 320, TN = 2,680; supervised TP = 2,930, FN = 70, FP = 975, TN = 2,025; and EM TP = 2,927, FN = 73, FP = 2,236, TN = 764. Recall is comparable across all three (97.3–97.7%), but the trained schemes increasingly sacrifice precision and specificity for that recall: supervised precision falls to 75.0% and specificity to 67.5%; EM falls to 56.7% precision and a very low 25.5% specificity. The EM run also estimates a much larger match prior (0.0071 versus the fixed 0.0001), which, together with EM-fitted m/u, drives the gross over-matching. The untrained default thus yields the best F1 (93.59%) on this duplicate-bearing population, reinforcing that the choice of m/u strongly affects the measured operating point and that reported figures must be tied to the exact parameter configuration used.

8.3 Decision-threshold and address-weight sensitivity

A grid search over the decision threshold τ and the address comparison weight was run on the same 5,000-record / 15,000-query confusion-matrix population. For each address weight the pipeline was scored once (all candidate pairs emitted at threshold zero) and each candidate decision threshold applied afterwards, giving 30 operating points for a small computational cost. The two findings below concern τ (where tuning helps) and the address weight (where it does not, on this data set).

Raising the decision threshold improves F1. At the default (full-strength) address comparison, increasing τ from 0.85 to 0.95 improves the F1 while keeping accuracy essentially flat. Because recall was already near its ceiling, the higher threshold removes the weakest false positives with only a modest recall penalty. Table 10 reports the trade-off.

Table 10: F1 versus decision threshold at full address weight on the 5,000-record confusion-matrix population (15,000 queries; artifact `results/erwhitepaper/f1_sweep_results.json`).

τ	Accuracy	Precision	Recall	Specificity	F1
0.85	99.28%	99.51%	99.41%	99.02%	99.46%
0.87	99.26%	99.52%	99.37%	99.04%	99.44%
0.90	99.24%	99.52%	99.34%	99.04%	99.43%
0.92	99.24%	99.52%	99.34%	99.04%	99.43%
0.95	99.52%	99.94%	99.34%	99.88%	99.64%

At $\tau = 0.95$ the confusion matrix is TP = 9,934, FN = 66, FP = 6, TN = 4,994: false positives fall from 49 to 6 (precision 99.51% to 99.94%, specificity 99.02% to 99.88%) while recall falls only

from 99.41% to 99.34% (F1 99.46% to 99.64%). The improvement is modest but consistent with the analysis above: in a regime where recall has headroom, the decision threshold is an effective precision lever and should be tuned on a validation set rather than fixed at 0.85.

Weakening the address comparison did not help. The intuition that address is too volatile to trust as an identity signal was tested by scaling the address agreement probabilities down ("weakening" the comparison) and re-running the sweep for strengths 0.6–0.9 (against the full-strength 1.0). On this data set the approach did not improve performance: after threshold tuning, every weakened configuration topped out at roughly 98.5% F1, well below the 99.64% reached by the full-strength address comparison at $\tau = 0.95$. The reason is that the address field here is genuinely informative: 30% of addresses are missing independently, and the addresses that do exist are long and highly discriminative, so address agreement is valuable evidence for correctly rescuing the close same-entity queries. Downgrading that evidence cost more true positives than it suppressed false ones.

This result is intentionally reported as a negative and should not be generalised. Its scope is limited to the synthetic Canadian population and its mutation model, where addresses are strongly identifying. There may be regimes in which weakening a volatile field is beneficial — for example, addresses that change frequently between legitimate records of the same entity, noisy or generic addresses (unit-only entries, post-office boxes), or populations with many address collisions in high-density buildings. Establishing whether and exactly when a weak-address weighting is viable requires controlled experiments across data sets with tunable address volatility; the implementation foresees this and `experiment_f1_sweep.py` plus `weaken_comparison` make such a study straightforward to run. That investigation is left as future work.

9 Non-Synthetic Replication: NC Voter Data

To test whether the pipeline behaves on real records rather than synthetic ones, we replicated the evaluation on North Carolina voter-registration data [18]. A statewide export was filtered to the fields relevant here — first and last name, birth year (the export carries no full birth date and no email) — yielding about 9.2 million records, from which a deterministic uniform sample of 5,000 was used. Because the public registration file is already heavily de-duplicated and contains no labelled duplicate pairs, authentic duplicate registrations are unavailable. We therefore apply a synthetic mutation model to the real records, analogous to the synthetic Canadian data, to create realistic noisy duplicates: names receive one-character typos, addresses are altered or omitted, and birth years occasionally shift by one. This yields genuine record linkage with noise on a real-world schema: positive queries are mutated duplicates whose clean base record is in the index and must be linked across the noise, and negative queries are mutated versions of held-out records that have no correct match in the index.

Using the same Splink configuration as the synthetic experiments (default m/u, top-20 blocking, $\tau = 0.85$; the email comparison is omitted because the data carries no email), FAISS recovered the clean base record for a mutated duplicate with 82.6%, 85.6%, and 88.0% recall at $k = 5, 10, 20$. Mutated-duplicate resolution produced TP = 1,284, FN = 216, FP = 0, TN = 1,500, an F1 of 92.2% (recall 85.6%, precision 100%). The false negatives align closely with the $\approx 12\%$ of mutations whose base record fell outside the top 20, so on this data the blocking stage is the binding constraint. The threshold/address-weight sweep again reproduced the synthetic finding: the full-strength address comparison at $\tau = 0.85$ reached F1 92.2%, while weakening the address comparison to 0.8 fell to 89.4%.

Because $k = 20$ still missed a nontrivial fraction of mutated candidates, we expanded the blocking size to $k = 50$ and $k = 100$ and measured both linkage quality and its cost, as summarised in Table 11. Larger k raises blocked-base recovery from 88.0% ($k = 20$) to 90.9% ($k = 50$) and 92.6% ($k = 100$), and end-to-end recall from 85.6% to 87.5% and 88.4%. The gains are monotonic but strongly diminishing (only +28 and then +14 true positives), while precision stays $\approx 100\%$ (exactly one false positive at $k = 100$), confirming the linkage model is robust to candidate inflation on this schema. Crucially, the recall gains come from the blocking stage rather than better linkage: even at $k = 100$ the end-to-end recall (88.4%) remains well below the blocking recall (92.6%), so the Splink stage — which discards a retrieved base below the threshold — is still the binding constraint, exactly as in the F1-budget analysis. The measured time impact on the Splink stage is modest: batched Splink scoring took 1.0 s at $k = 20$, 1.5 s at $k = 50$, and 2.4 s at $k = 100$ (about a 2.4× rise) as the candidate set grows roughly fivefold, while FAISS blocking time stayed near 50 s.

Table 11: Effect of blocking size on mutated NC-voter linkage (3,000 positives, 1,500 negatives, $\tau = 0.85$; artifacts `results/erwhitepaper/ncvoter/results_resolution.json` and the k=50, k=100 variants). Larger k recovers more mutated duplicates, but gains are diminishing and the Splink stage remains the binding constraint; Splink scoring time grows with the candidate set while blocking time stays effectively constant.

k	Blocking recall	End-to-end recall	Precision	F1	Splink scoring time
20	88.0%	85.6%	100.0%	92.2%	1.0 s
50	90.9%	87.5%	100.0%	93.3%	1.5 s
100	92.6%	88.4%	99.9%	93.8%	2.4 s

These figures are more informative than exact self-resolution but remain bounded by the source. First, the registration file is still well de-duplicated, so every positive is a synthetic mutation of an otherwise distinct base record rather than a genuine duplicate registration; the mutation model proxies noisy duplicate intake (typos, address churn, missing fields) but cannot reproduce real voter turnover or duplicate-file scenarios. Second, the export contains only a birth year and no email, so the date-of-birth and email comparisons — two identity signals in the synthetic experiments — are only weakly tested: date-of-birth errors are not properly exercised, and email is absent entirely. The replication therefore confirms the pipeline on a real schema with injected noise while leaving authentic duplicate, full-DOB, and email scenarios to the data sets discussed in Section 10.

10 Further Research

10.1 Alternative embedding engines

The current MiniLM encoder is a useful baseline, but it is not specialized for entity resolution or structured rows. Candidate alternatives include:

- larger sentence-transformer models for stronger semantic representations;
- multilingual encoders for names and addresses across language communities;
- character n-gram or TF–IDF vectors for typo-sensitive retrieval;
- domain-adapted contrastive encoders trained on positive and hard-negative person pairs;
- graph-based universal tabular embeddings such as those proposed by Franz et al. [7]; and

- hybrid retrieval that combines dense similarity with lexical or field-specific indexes.

The comparison should measure not only final linkage quality but also whether the embedding preserves the true match in the top 20. A particularly promising approach is multi-view retrieval: learn separate views for identity fields, contact fields, and address fields, then combine their scores before Splink. This could reduce the tendency of a long address to dominate a short but highly identifying date of birth.

These directions are nevertheless worth tempering against the measured F1 budget. On this paper's primary data set the default top-20 blocking recall is already 99.88% (rising to 99.95% with the compact serialization), which caps end-to-end recall. Because the measured end-to-end recall (99.41%) sits well below that ceiling, the recall that a better blocker could add is at most 0.12% (default serialization), and even a perfect blocker would raise the F1 from 99.46% toward about 99.69% — a gain of roughly two tenths of a point. In other words, alternative embeddings and multi-view retrieval are unlikely to move F1 materially while top-k blocking recall is already near-saturated. Their value would instead emerge in regimes where blocking recall is the binding constraint: much larger and noisier populations, or retrieval configurations that no longer hold near-perfect recall at a small k. Absent such a regime, the higher-impact research target remains the Splink linkage stage, which currently gives up about 0.47% of recall between blocking and the final decision and is where the precision-recall trade-off can actually be tuned (for example through the threshold and m/u calibration experiments above).

10.2 Canopy-based parameter training

The Splink parameters trained in this paper — whether by unsupervised expectation maximisation over the reference population (Section 8.1) or by supervised calibration over labelled pairs — are fitted on candidate pairs generated by exact-equality blocking on the `first_name` and `date_of_birth` keys. Embedding blocking offers an alternative candidate-generation strategy for training: instead of value-based equality, candidate pairs could be produced by dense-neighbour clustering of the same row embeddings used for online blocking. The classic recipe is *canopy clustering*, in which a cheap distance measure places each record into overlapping candidates using two thresholds (a wide capture radius and a tighter one), after which an expensive similarity is applied only within the tighter canopies [10]. Dense top-k retrieval is a direct analogue: each query's retrieved neighbourhood is a canopy, so the online blocker in this paper is already canopy-shaped, and the same construction could supply training pairs.

The argument for doing so is that canopy/embedding candidates are density-derived rather than attribute-derived, so an EM fit over them resembles the collision structure of a real retrieval workload more closely than value-equality blocks do. This direction is well supported in the entity-resolution literature: embedding models have been used to build the candidate sets on which later matchers and training pipelines operate [5], similarity-based (including vector) blocking has been evaluated directly against exact-match blocking [11], pre-trained-embedding blocking variants have been benchmarked head-to-head against each other and against a vector-based state of the art [4], and learned blocking functions selected from labelled seeds can be viewed as data-driven analogues of canopy selection [12]. In all of these the embedding blocker feeds a downstream matcher, and the same feed-forward relationship applies to training the matcher's own parameters.

This is therefore a concrete next step: replace the value-equality training blocks with a `block_id` derived from FAISS clusters (the store already exposes the vector space used for online blocking), and re-fit the Splink m/u and prior on those candidate pairs. This canopy-style two-phase design, and its reuse of the same index for batch deduplication and insertion-time duplicate prevention,

is developed experimentally in a companion paper on batch deduplication with a shared vector canopy index [28]. The approach is not yet implemented or measured in this paper, so it is flagged as future work rather than a reported result. Critically, the calibration caveats in the Calibration Paradox companion paper still apply: candidate pairs produced by embedding neighbourhoods are even more resemblance-biased than value-equality collisions, so a freely fitted prior over them is expected to drift upward *more* than the 7.2×10^{-3} observed with value blocking, and any such prior must be re-anchored and τ re-tuned jointly.

10.3 Continuous matcher scores and the Fellegi–Sunter weights

The Splink comparisons in this paper produce *discrete* levels, each with its own m/u probabilities and therefore its own match weight. Learned matchers, by contrast, return a *continuous* score — a deep-network probability (DeepMatcher, Ditto) or an embedding cosine – and the question is how to apply Fellegi–Sunter m/u parameters to such scores rather than to categorical levels.

The standard recipe treats a continuous score x as a comparison level of infinite resolution and forms the likelihood ratio

$$\frac{m(x)}{u(x)} = \frac{f(x \mid M)}{f(x \mid U)},$$

which feeds the same Fellegi–Sunter log-odds as a discrete level weight. The two densities must be estimated from data, by fitting a normal-mixture model to the score distribution, by calibration against labelled pairs, or by EM — a standard Fellegi–Sunter estimation problem. Winkler [21] reviews how comparators' scores are mapped to weights, including the mixture-model estimation of m/u that he and Jaro introduced; Christen's survey [23] describes these techniques only via those primary sources.

The modern, directly relevant line applies the same idea to learned matchers: the end-to-end survey of Christophides et al. [14] positions learned matchers and probabilistic (Fellegi–Sunter) linkage as stages of one entity-resolution pipeline, alongside its blocking stage, and Sadinle and Fienberg [15] generalise the Fellegi–Sunter construction to the multiple-record setting, fitting match patterns from the comparison data by mixture maximum likelihood. Together these yield the practical recipe for this paper's setting: treat the matcher score as the comparison, estimate $f(x \mid M)$ and $f(x \mid U)$ (by mixture modelling as in [21], by calibration against labelled pairs, or by EM; cf. the survey [23]), form $w(x) = \log_2[f(x \mid M)/f(x \mid U)]$, and add it to the FS log-odds exactly like a level weight, with the match prior λ unchanged.

This matters for two reasons in the present work. First, the deep-matcher literature — DeepMatcher [13], DeepER [5], Ditto [16], and large-language-model matchers [17] — all operate on the matching step, deciding match/no-match for the record pairs supplied to them; none addresses the retrieval-side question of whether a far-apart duplicate is a candidate at all. That is the gap the gap-weighted, capture-date-conditional fusion of this paper targets, and it is orthogonal to how the matcher's score is converted into a weight. Second, the m/u calibration caveats already established here apply a fortiori to continuous scores: a score distribution fitted on resemblance-biased candidate pairs (as from dense-neighbour blocking) will inherit the same prior drift as the value-blocking case, so the calibration lesson — anchor the prior and re-tune τ — carries over unchanged.

10.4 Replication on non-synthetic data

The central experiments in this paper are measured on synthetically generated Canadian person records (Faker with a custom Canadian address provider). Synthetic data is valuable because it supplies controlled missingness, exact ground truth, and repeatable duplicate rates, but it embeds

assumptions about name distributions, address formats and volatility, email conventions, and how errors arise between records of the same entity. The replication on the North Carolina voter data (Section 9) is a first step toward real data, but, as noted there, it is a near-best-case scenario that does not fully exercise cross-record duplicates, naming variation, or birth-date errors. Whether the findings hold more broadly with real data is therefore still an open empirical question, and the experiments should be replicated on non-synthetic, labelled record-linkage data. The replication should run the identical pipeline and protocols — same top-k FAISS blocking, same comparisons and decision machinery, the confusion-matrix three-query protocol, and the trained/untrained/supervised m/u, threshold, and address-weight sweeps — and compare the resulting confusion matrices, F1, and the conclusions drawn from them.

The natural test beds are public benchmark record-linkage data sets with known ground truth. These include the Cora citation data [10], the FEBRL restaurant data [23], and the product and citation pairs from the Abt–Buy, Amazon–Google, and DBLP–ACM benchmarks used in the deep-matcher literature [13] (data sets: https://dbs.uni-leipzig.de/research/projects/benchmark-datasets-for-entity-resolution/). Where a data set lacks the fields used here (date of birth, address, email), the comparison should be restricted to the fields it does provide, or the pipeline should be adapted to the available schema while keeping the methodology fixed. While the benchmark data sets above provide authentic duplicate pairs with ground truth, public registration data such as the North Carolina voter file is a useful but limited test bed: it is heavily de-duplicated by the source and typically lacks a full date of birth, so it contains few authentic duplicate registrations and cannot exercise birth-date errors. The mutation-model replication in Section 9 supplies a noisy-duplicate proxy on a real schema, but authentic duplicates and full-DOB comparisons still require the labelled benchmark or corporate data discussed below.

Such replication serves several specific purposes. First, it tests whether the address field is as informative as the synthetic data suggests: public and production datasets expose real address volatility and generic or unit-only addresses, so the finding that simply weakening the address comparison did not help may not transfer. Second, real data can reveal whether retraining m/u on the collection itself behaves like the lab setting, where the untrained default outperformed the trained variants, or whether genuinely noisy real duplicates make calibration beneficial. Third, the recall distributions and blocking ceiling measured at $k = 20$ in Section 10 may differ when names collide at realistic rates, changing where the F1 budget lies between the blocking and linkage stages. Finally, replication must handle missing values, inconsistent postal/date formats, and multilingual names, which the synthetic generator controls but real data exposes. Privacy considerations apply because person records are personally identifying; labels and ground truth should be obtained from datasets that are published for research or from de-identified data with controlled access. The goal is not to certify a specific score but to confirm which qualitative conclusions — which levers move F1 and which do not — survive contact with real data.

Beyond public benchmarks, real-world performance is best assessed on corporate data: production master records that an organisation genuinely needs to de-duplicate or match, such as customer, patient, employee, or product records, ideally with a human-validated sample of true duplicates. Public data sets are usually small, masked, or already de-duplicated, whereas corporate data carries the actual error profile the deployment will face — real naming conventions, address churn, partial or corrupted fields, and the true duplicate rate of the workload. That allows the pipeline to be calibrated and its F1 reported against the workload it will actually serve, which is the strongest test of real-world performance. The constraints are access and privacy: matching personnel must have legitimate authority, data transfers need privacy controls, ground-truth labelling should be reviewed for accuracy, and only the minimum necessary evidence should be logged. A mid-size corporate master (tens of thousands to a few million records) is typically large enough

to expose distributional differences that the small synthetic and public sets miss while remaining small enough to run the experiments directly. Such partnerships make it possible to validate both the quantitative results and the qualitative conclusions about which levers, such as the decision threshold, matter most in practice.

10.5 Calibration and deployment

Future work should estimate Splink parameters from labelled operational samples, quantify probability calibration, and introduce drift monitoring for names, domains, postal formats, and address conventions. Privacy-preserving evaluation is also important: synthetic data is useful for development, but production validation must control access to personally identifying information and log only the minimum necessary evidence.

10.6 Parsing and segmentation

The pipeline currently consumes already-structured fields (the synthetic generator and the NC-voter export supply separate name and address columns), but real-world ingestion is frequently unfielded raw strings. Research should quantify the impact of name and address parsing and segmentation — turning a raw name or block address into first/middle/last/suffix and street/unit/city/state/postcode components and normalizing them — on both the accuracy and the resource usage of the pipeline. On the accuracy axis, the questions are (i) how segmentation errors propagate through the row embedding into blocking recall and then into linkage F1 and calibration: a mis-segmented name or a wrongly split address degrades both the embedding and the string comparisons Splink relies on, and it changes the non-match agreement rate (u) that the linkage observes; and (ii) how the choice of parser — regex heuristics, a statistical parser such as libpostal, a neural token tagger, or an instruction-tuned LLM — changes those outcomes, measured against deliberately unfielded or segmentation-noisy inputs. On the resource axis, the research should cover parsing cost per record (CPU/GPU time, memory, and per-query latency) and, in particular, where a staged strategy lands in the pipeline's sub-10 ms query budget: a fast deterministic/library tier for the bulk of well-formed records, a lightweight transformer tagger for names, and an LLM reserved for the messy residual tail, whose latency and cost then trade against downstream blocking and linkage. Because segmentation sits upstream of embedding, blocking, and linkage, its errors amortize across every later stage; quantifying that amplification, and the accuracy-versus-resource frontier across parser tiers, is a concrete high-value next step.

11 Conclusion

The implemented architecture combines a fast vector retrieval layer with an interpretable probabilistic linkage layer. FAISS narrows a 50,000-record population to 20 candidates, while Splink compares identity, contact, and address fields under a configurable decision threshold. Persisting the index and metadata makes the workflow operationally reusable. The most important next steps are to train and calibrate Splink parameters on labelled pairs and benchmark alternative row and tabular embedding engines using blocking recall as a first-class metric.

The measured experiments also isolate which control levers move the F1 across the validation population and which do not. Three levers had a clear effect. The decision threshold τ is the cheapest and most effective precision lever: because recall already sat near its ceiling, raising τ from 0.85 to 0.95 on the confusion-matrix population lifted F1 from 99.46% to 99.64% (precision 99.51% to 99.94%, specificity 99.02% to 99.88%) at a small recall cost. The row serialization

strategy also matters: the compact representation raised top-20 blocking recall to 99.95% and F1 to 99.73%, confirming that retrieval recall is a first-order lever for end-to-end F1. So is the blocking size k, which sets an upper bound on end-to-end recall; the default $k = 20$ already achieves 99.88% blocking recall, so increasing k buys only a marginal recall gain while adding retrieval cost. Two levers did not help on this data set. Weakening the address comparison (scaling its agreement probabilities down by factors 0.6–0.95) never improved F1, topping out near 98.5% rather than the 99.64% reached at full address strength, because address agreement is genuinely informative for the close same-entity queries here. Likewise, retraining m/u on the synthetic population — whether supervised or by expectation maximisation — over-matched and lowered F1 relative to the untrained default: calibration only helps when it is driven by labelled, operationally representative pairs, which the synthetic reference does not contain.

A Deriving the Decision Threshold τ

The decision threshold τ should be treated as a statistical boundary rather than an arbitrary heuristic. The standard algorithms for deriving it are grouped here by whether labelled ground truth is available.

A.1 With labelled ground truth (supervised derivation)

If a labelled validation set of true matches and non-matches exists, classical optimisation applies.

Expected-cost minimisation. If a false positive carries a business cost C_{FP} and a false negative a cost C_{FN}, then, assuming correct decisions carry no cost and the posterior $P(M \mid \gamma)$ is well calibrated, decision theory states that the expected loss is minimised by declaring a pair a match exactly when its posterior match probability satisfies $P(M \mid \gamma) \geq \tau^*$, with

$$\tau^* = \frac{C_{FP}}{C_{FP} + C_{FN}} .$$

For example, if merging two distinct accounts (C_{FP}) is twenty times more costly than failing to link two records of the same person (C_{FN}), then $\tau^* = 20/(20 + 1) \approx 0.95$. This is the standard Bayes decision-theoretic threshold; the optimality criterion in Fellegi–Sunter [1, 19], by contrast, is error-bound based: for fixed upper bounds on the false-match and false-nonmatch error rates, minimise the expected number of pairs left undecided, yielding the two-threshold (link / possible-link / non-link) rule rather than a single cost ratio. Two caveats apply to the cost formula: it presumes costs that are constant across pairs and zero cost for correct decisions (if a correct merge itself has an operational cost, or costs vary by pair, the threshold shifts and is no longer a single global value); and it is optimal per pair only when pair decisions are independent — under transitivity and one-to-one constraints it is a heuristic operating point (see the transitivity rule below). With labels in hand, a more robust alternative is to sweep τ on the validation set to minimise empirical cost directly, which does not depend on calibration.

F1 maximisation. Alternatively one maximises the empirical F1 score by sweeping candidate thresholds:

$$\tau^* = \underset{\tau \in [0.5, 0.999]}{\arg\max} \frac{2 \cdot \text{Precision}(\tau) \cdot \text{Recall}(\tau)}{\text{Precision}(\tau) + \text{Recall}(\tau)} .$$

Precision-recall trade-offs and threshold selection are standard practice for skewed classification [22, 23]. The threshold and address-weight sweeps in this paper (Table 10 and the NC-voter experiments) instantiate this supervised rule on a labelled validation set.

A.2 Without labelled ground truth (unsupervised derivation)

When labels are absent, empirical precision and recall cannot be measured directly, and the boundary must be inferred from the data.

Expected-F1 maximisation. If the match probabilities $p_j = P(M \mid \gamma_j)$ estimated by the fitted model (e.g., by expectation maximisation) are well calibrated, the expected counts at any threshold τ can be computed without labels. With $D_\tau = \{j : p_j \geq \tau\}$ the pairs classified as matches, the expected true positives, false positives, and false negatives are

$$E[TP(\tau)] = \sum_{j \in D_\tau} p_j, \qquad E[FP(\tau)] = \sum_{j \in D_\tau} (1 - p_j), \qquad E[FN(\tau)] = \sum_{j \notin D_\tau} p_j,$$

so the plug-in F1 computed from these expected counts (an approximation valid for large $|D_\tau|$ via the delta method, and treating pairs as independent Bernoulli draws) is

$$E[F_1(\tau)] = \frac{2 \sum_{j \in D_\tau} p_j}{|D_\tau| + \sum_j p_j},$$

where $\sum_j p_j$ is the total expected number of matches in the candidate-pair set S. One then selects $\tau^* = \arg\max_\tau E[F_1(\tau)]$ over a dense grid. This is a direct use of the calibrated posterior probabilities produced by the mixture/EM fitting of the Fellegi–Sunter model as reviewed in [21]; it approximates the cost-based rule when the two error types are weighted equally.

Gaussian-mixture valley detection. The Fellegi–Sunter weights $w_j = \log_2 R(\gamma_j)$ are typically bimodal: a large peak of low-weight non-matches and a smaller high-weight match peak. Fitting a two-component Gaussian mixture, in the weight-fitting tradition of [21], to the weights yields densities $f_U(w)$ and $f_M(w)$ for the non-match and match components; the threshold is placed at the intersection

$$p\, f_M(w^*) = (1 - p)\, f_U(w^*),$$

with p the prior match probability, and the weight w^* converted back to a probability via the Bayes relation $\tau^* = \left(1 + \frac{1-p}{p}\, 2^{-w^*}\right)^{-1}$. This places the boundary at the statistical "valley" separating the two populations; frequency-based and decision-rule refinements of the Fellegi–Sunter weights also assume such well-separated weight distributions [20].

Graph-transitivity consistency. Linkage is expected to be transitive (if A matches B and B matches C, then A matches C). For a candidate τ, build a graph with an edge between records i, j when $p_{ij} \geq \tau$, count transitivity violations (e.g., A connected to B and B to C but A not to C), and select the τ that minimises violations while maximising cluster cohesion, for example the silhouette score of the resulting connected components. Record-linkage clustering under transitive closure is standard in the field [23].

B Tuning the Match Prior λ

The calibration-paradox analysis (Section 8.1) showed that retraining m/u produced worse results than leaving them untrained, and decomposed the effect into prior coupling and a genuine m/u residual. The dominant, addressable part is the prior: EM's free estimate of the match prior λ (here 0.0071 versus the default 0.0001) inflated the match probabilities and produced gross over-matching. This appendix outlines an algorithm that tunes the prior and the decision threshold together, which is the fix the paradox calls for. Let $\lambda = \textit{probability\_two\_random\_records\_match}$ and τ be the decision threshold.

Step 0 (baseline). Set $\lambda_0 = 0.0001$ (the Splink default).

Step 1 (estimate a principled prior; do not use EM's free prior).

- *With labels:* $\lambda_{\text{est}} = \frac{\text{proportion of candidate pairs that are true matches}}{\text{blocking recall}}$, i.e. the base match rate in random-record-pair space rather than within the blocked candidate set.
- *Without labels:* use Splink's estimator `estimate_probability_two_random_records_match`, called with the blocking rules and a `recall`; it divides the expected matches by the blocking coverage. If no defensible match-rate assumption exists, keep $\lambda_0 = 0.0001$.

Step 2 (fit m/u with the prior pinned). Run Splink EM with `fix_probability_two_random_records_match = True` so that EM estimates m/u only, or keep the untrained defaults. This is the key divergence from the paradox setup: m/u may change while λ stays anchored.

Step 3 (choose (λ, τ) jointly on validation). Grid over $\lambda \in \{\lambda_0, \lambda_{\text{est}}\} \times 10^{-k}$ and $\tau \in [0.5, 0.999]$; for each cell score all pairs and evaluate the objective. *With labels:* decision cost $C_{FP} \cdot FP + C_{FN} \cdot FN$ (or F1), selecting the minimiser/argmax. *Without labels:* the expected-F1 or expected-cost from the calibrated posteriors, or the GMM-valley threshold, as defined in Appendix A.

Step 4 (coherence guard). Reject solutions whose *mean posterior probability of non-match pairs* is high (EM's was about 0.53); this is the diagnostic that separates prior coupling from a genuine m/u residual. Prefer a smaller λ, and verify that false positives and false negatives move monotonically with τ around the chosen λ.

Step 5 (freeze and report). Lock (λ^*, τ^*) on validation and measure once on a held-out test set, reporting λ, τ, F1, and the false-positive/false-negative/cost outcome.

Why this addresses the paradox: (i) λ is anchored — the default or the blocking-adjusted estimate — instead of letting EM inflate it to 0.0071, which recovered EM's F1 from about 0.83 to about 0.95 in our data; (ii) λ and τ are tuned together, matching the coherent-bundle rule that no single element is changed in isolation; (iii) the score-shift guard distinguishes prior coupling from a real model-fitting problem before the numbers are trusted. Scope note: the blocking-adjusted step is Splink-native; the joint grid and guard are the contribution here. This appendix addresses the *prior*; the remaining genuine residual (trained m/u slightly below the defaults, 0.955 versus 0.990) is a model-fitting question not solved by prior tuning.

Use of Artificial Intelligence

During the preparation of this work the author used an AI coding and writing assistant — the DeepSeek V4 Flash 0731 large language model accessed through an interactive coding agent — to assist with drafting and editing the manuscript, authoring and debugging the code that implements and evaluates the pipeline, and running the experiments and automation that produced the reported numbers. The tool was not listed as an author, as it cannot take responsibility for the work. All numerical results reported in this paper were obtained by executing the repository code and were independently re-checked for accuracy. Because generative assistants can hallucinate citations, every bibliographic entry in the reference list was re-verified by the author against the publisher, Crossref, DBLP, or arXiv records in August 2026; the record for each entry states its DOI or stable archive identifier, and any entry that could not be verified was removed or replaced. Every claim, table, figure, result, and conclusion was thoroughly reviewed and verified by the author, who takes full responsibility for the content of this document, including any remaining errors and/or omissions.

References

[1] Ivan P. Fellegi and Alan B. Sunter. A theory for record linkage. Journal of the American Statistical Association, 64(328):1183–1210, 1969. https://doi.org/10.1080/01621459.1969.10501049.

[2] Robin Linacre, Sam Lindsay, Theodore Manassis, Zoe Slade, Tom Hepworth, Ross Kennedy, and Andrew Bond. Splink: Free software for probabilistic record linkage at scale. International Journal of Population Data Science, 7(3), 2022. https://doi.org/10.23889/ijpds.v7i3.1794. Software repository: https://github.com/moj-analytical-services/splink.

[3] Tianshu Wang, Hongyu Lin, Xianpei Han, Xiaoyang Chen, Boxi Cao, and Le Sun. Towards universal dense blocking for entity resolution. arXiv preprint arXiv:2404.14831, 2024. https://arxiv.org/abs/2404.14831. Code repository: https://github.com/tshu-w/uniblocker.

[4] Alexandros Zeakis, George Papadakis, Dimitrios Skoutas, and Manolis Koubarakis. Pre-trained embeddings for entity resolution: An experimental analysis. Proceedings of the VLDB Endowment, 16(9):2225–2238, 2023. https://doi.org/10.14778/3598581.3598594.

[5] Muhammad Ebraheem, Saravanan Thirumuruganathan, Shafiq Joty, Mourad Ouzzani, and Nan Tang. Distributed representations of tuples for entity resolution. Proceedings of the VLDB Endowment, 11(11):1454–1467, 2018. https://doi.org/10.14778/3236187.3236198.

[6] Tymoteusz Strojny and Maciej Beręsewicz. BlockingPy: Approximate nearest neighbours for blocking of records for entity resolution. SoftwareX, 34:102583, 2026. https://doi.org/10.1016/j.softx.2026.102583. Software repository: https://github.com/ncn-foreigners/BlockingPy.

[7] Astrid Franz, Frederik Hoppe, Marianne Michaelis, and Udo Göbel. Universal embeddings of tabular data. arXiv:2507.05904v1, 2025. https://arxiv.org/abs/2507.05904v1.

[8] Liane Vogel, Kavitha Srinivas, Niharika D'Souza, Sola Shirai, Oktie Hassanzadeh, and Horst Samulowitz. Towards universal tabular embeddings: A benchmark across data tasks. arXiv:2604.21696v1, 2026. https://arxiv.org/abs/2604.21696v1.

[9] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. IEEE Transactions on Big Data, 7(3):535–547, 2021. https://doi.org/10.1109/TBDATA.2019.2921572. Preprint: https://arxiv.org/abs/1702.08734.

[10] Andrew McCallum, Kamal Nigam, and Lyle H. Ungar. Efficient clustering of high-dimensional data sets with application to reference matching. In Proceedings of the Sixth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), pages 169–178, 2000. https://doi.org/10.1145/347090.347123. Data set (Cora): https://www.openicpsr.org/openicpsr/project/100859/version/V1/view and https://hpi.de/naumann/projects/repeatability/datasets/cora-dataset.html.

[11] Saravanan Thirumuruganathan, Han Li, Nan Tang, Mourad Ouzzani, Yash Govind, Derek Paulsen, Glenn Fung, and AnHai Doan. Deep learning for blocking in entity matching. Proceedings of the VLDB Endowment, 14(11):2459–2472, 2021. https://doi.org/10.14778/3476249.3476294.

[12] Wei Zhang, Hao Wei, Bunyamin Sisman, Xin Luna Dong, Christos Faloutsos, and David Page. AutoBlock: A hands-off blocking framework for entity matching. In Proceedings of the 13th ACM International Conference on Web Search and Data Mining (WSDM), pages 744–752, 2020. https://doi.org/10.1145/3336191.3371813.

[13] Sidharth Mudgal, Han Li, Theodoros Rekatsinas, AnHai Doan, Youngchoon Park, Ganesh Krishnan, Rohit Deep, Esteban Arcaute, and Vijay Raghavendra. Deep learning for entity matching: A design space exploration. In Proceedings of the 2018 International Conference on Management of Data (SIGMOD), pages 19–34, 2018. https://doi.org/10.1145/3183713.3196926.

[14] Vassilis Christophides, Vasilis Efthymiou, Themis Palpanas, George Papadakis, and Kostas Stefanidis. An overview of end-to-end entity resolution for big data. ACM Computing Surveys, 53(6):127:1–127:42, 2021. https://doi.org/10.1145/3418896.

[15] Mauricio Sadinle and Stephen E. Fienberg. A generalized Fellegi–Sunter framework for multiple record linkage with application to homicide record systems. Journal of the American Statistical Association, 108(502):651–660, 2013. https://arxiv.org/abs/1205.3217.

[16] Yuliang Li, Jinfeng Li, Yoshihiko Suhara, AnHai Doan, and Wang-Chiew Tan. Deep entity matching with pre-trained language models. Proceedings of the VLDB Endowment, 14(1):50–60, 2020. https://doi.org/10.14778/3421424.3421431.

[17] Ralph Peeters, Aaron Steiner, and Christian Bizer. Entity matching using large language models. arXiv preprint arXiv:2310.11244, 2023. https://arxiv.org/abs/2310.11244.

[18] North Carolina State Board of Elections. Voter registration data (public data download). https://www.ncsbe.gov/results-data/voter-registration-data.

[19] William E. Winkler. Improved decision rules in the Fellegi–Sunter model of record linkage. In Proceedings of the Section on Survey Research Methods, American Statistical Association, pages 274–279, 1993.

[20] William E. Winkler. Frequency-based matching in the Fellegi–Sunter model of record linkage. Statistical Research Division, U.S. Bureau of the Census, Research Report Series RR2000/06, 2000. https://www.census.gov/library/working-papers/2000/adrm/rr2000-06.html.

[21] William E. Winkler. Overview of record linkage and current research directions. Statistical Research Division, U.S. Bureau of the Census, Research Report Series RRS2006/02, 2006. https://www.census.gov/library/working-papers/2006/adrm/rrs2006-02.html.

[22] Jesse Davis and Mark Goadrich. The relationship between precision-recall and ROC curves. In Proceedings of the 23rd International Conference on Machine Learning (ICML), pages 233–240, 2006. https://doi.org/10.1145/1143844.1143874.

[23] Peter Christen. Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection. Springer, Data-Centric Systems and Applications, 2012. https://doi.org/10.1007/978-3-642-31164-2.

[24] Denis Robert. Why the decaying address weight cannot be absorbed into m/u. Technical note, Independent Researcher, 2026.

[25] Denis Robert. Time decay in Fellegi–Sunter record linkage: A literature survey. Technical note, Independent Researcher, 2026.

[26] Robin Linacre. Comment on Splink issue #3240 concerning piecewise temporal decay via comparison levels. GitHub, moj-analytical-services/splink, 2026. https://github.com/moj-analytical-services/splink/issues/3240#issuecomment-5309218620.

[27] Denis Robert. The calibration paradox in Fellegi–Sunter processes. Technical report, Independent Researcher, 2026.

[28] Denis Robert. Batch deduplication and insertion-time prevention with a shared vector canopy index. Technical report, Independent Researcher, 2026.